Injection Vulnerability (Malicious Code Injection Error)

Concept

Injection is a vulnerability that occurs due to a failure to filter untrusted input data. When you transmit unfiltered data to a database (e.g., SQL injection), to a browser (XSS vulnerability), to an LDAP server (LDAP Injection vulnerability), or to any other location, the problem is that an attacker can insert malicious code to cause data leaks and gain control of the client's browser.

Consequence

Injection can lead to data loss, disruption or disclosure to third parties, loss of accounts, or denial of access. Sometimes, injection results in the hijacking of control.

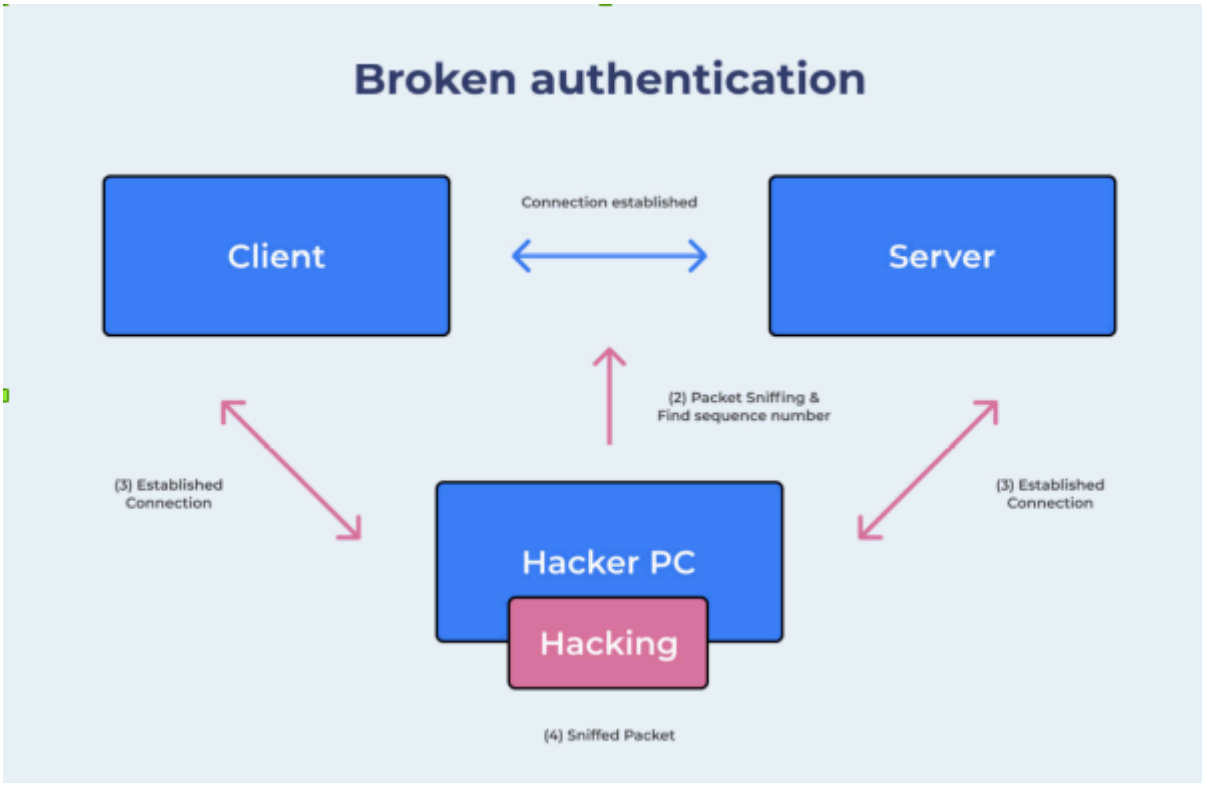
Prevention methods

The fundamental principle in combating injection is to separate the data being sent from the direct execution statement and queries.

- There are mechanisms for checking and filtering input data, such as limiting size, removing special characters, etc.
- Use stored procedures in the database. Essentially, this involves putting SQL queries inside procedures (similar to functions in programming languages), with data passed to the procedure via parameters, thus separating the data from the code.
- Do not display any exceptions or error messages to prevent attackers from guessing the results.
- Set appropriate user permissions.
- Proactively use security vulnerability scanning tools.
- Back up your data regularly.

Broken Authentication

Concept



This vulnerability relates to user authentication issues and improperly implemented session management, resulting in weak management mechanisms. This allows hackers to crack passwords, keys, session tokens, or exploit other vulnerabilities to gain temporary or permanent access to user identities.

Attackers can easily find hundreds of thousands of common usernames and passwords, a list of default admin accounts, automated brute force tools, or dictionary attack toolkits. These are necessary conditions for carrying out an attack targeting this vulnerability. With access to a few accounts, or even just one admin account, a hacker can compromise the entire system. Depending on the nature of the system, this allows the hacker to commit various illegal acts such as money laundering, theft of assets and identities, and disclosure of sensitive information.

So what kind of system would be at risk of this vulnerability?

- The application allows hackers to carry out brute force attacks or other types of automated attacks. Simply put, our web application allows continuous requests to multiple APIs from the same IP address or attempts to access an account multiple times without any management mechanism, such as locking the account if such behavior occurs.
- Allows users to set weak, non-standard passwords. There is no mechanism to force changes to default passwords such as "Password1" or "admin/admin".

- The password recovery mechanism (in case users forget their password) is weak or insecure, such as the question-answering mechanism you often saw if you used Yahoo 7-8 years ago; this is actually a very poor solution at the present time.
- Storing passwords in plaintext without encryption, or using simple encryption algorithms or hash codes, makes them easily crackable.
- Lack of or ineffective two-factor authentication.
- Display Session IDs or key parameters directly within the URL's Params.
- There is no mechanism to automatically change Session IDs after a successful login.
- The session ID duration settings are incorrect.

Consequence

A hacker would need to gain access to several accounts, or even just one admin account, to infiltrate the system.

Depending on the system's purpose, Broken Authentication can allow money transfers and disclose highly sensitive information.

Prevention methods

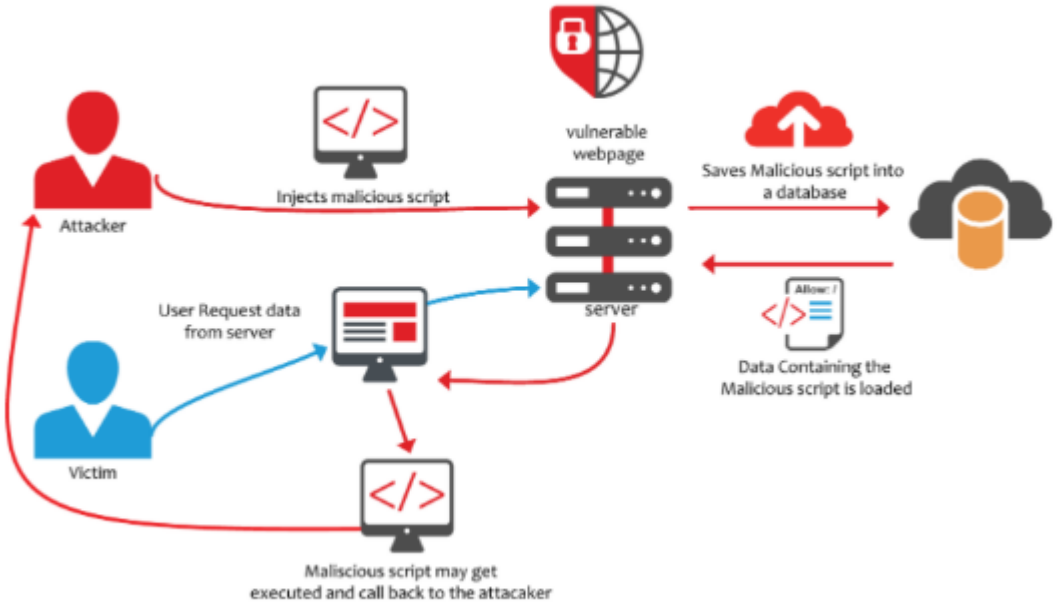
- Implement a two-factor authentication mechanism to combat automated attacks such as brute force.
- Check the security of passwords, preventing users from setting overly simple passwords such as those consisting only of numbers or letters. You can also check the passwords users set in the top 10,000 worst passwords.https://github.com/danielmiessler/SecLists/tree/master/Passwordsand do not allow these passwords to be set.
- Ensure that registration, account recovery, login failures (possibly due to incorrect password or username), or URL paths use the same messages returned to the user for all results. For example, when a user fails to log in due to an incorrect password, if the server returns the message "You entered the wrong password," an attacker can use this to know that the submitted username exists and simply brute-force the password until successful. Instead, the server should return the message "Invalid username or password," preventing the attacker from eliminating any possibility and making brute-force attacks much more complex.
- Limit the number of attempts or require a waiting period after a certain number of unsuccessful logins. This could involve locking the account (as Facebook currently does), or waiting 2-3 minutes before attempting to log in again. This

mechanism is quite common, similar to unlocking an iPhone where your device is disabled after a certain number of incorrect password attempts.

- It has a mechanism for generating, managing, and storing Session IDs that ensures security, with high difficulty and scrambling capabilities, set expiration times, and self-destruction after logging out.

XSS (Cross-Site Scripting) vulnerability

Concept



XSS (Cross-site Scripting) vulnerabilities are very common. Attackers inject script code into web applications. When this input is not filtered, it executes malicious code in the user's browser. Attackers can obtain users' cookies on the system or trick users into visiting malicious websites.

=> Mechanism: The hacker will enter script code into the Input field, then the server will save it to the database. When the user sends a request to the server, the server will load the data from the database to process it, and the script code will be executed. The hacker can then obtain the user's cookies or redirect the user to a more dangerous website.

Consequence

The script is executed on the user interface, allowing the hacker to perform tasks such as stealing credentials, sessions, or delivering malware to the victim.

Prevention methods

- Use frameworks that automatically detect XSS, such as Ruby on Rails, ReactJS, and Laravel.
- One simple web security measure is to avoid returning HTML tags to the user. This also helps protect against HTML Injection – a similar attack where hackers target HTML content – which, while not seriously damaging, is quite inconvenient for users. The usual solution is simply to encode all HTML tags.

Broken Access Control:

Concept

This is a common permission error in the system. Permission errors often result from a lack of automated error detection systems, ineffective testing methods, or inefficient test functions, leading to system leaks.

Consequence

Technically, hackers can gain access and have user or administrator privileges, or users with knowledge of system functions can call and execute these functions (e.g., adding, editing, or deleting records).

Prevention methods

- Restrict API and controller access to minimize damage.
- Implement access control mechanisms and enforce them across the entire application.
- JWT tokens should be disabled on the server when logging out.
- You should set up rules in the Model to manage operations with the database.

Sensitive Data Exposure:

Concept

This vulnerability relates to the crypto and resource aspect. Sensitive data must be encrypted at all times, including during transmission and storage – no exceptions are permitted. Credit card information and user passwords should never be transmitted or

stored unencrypted. Clearly, encryption and hashing algorithms are not a weak security measure. Furthermore, web security standards recommend the use of AES (256 bits or more) and RSA (2048 bits or more).

It should be noted that Session IDs and sensitive data should not be transmitted in URLs, and sensitive cookies should have a security flag.

Consequence

Releasing sensitive information can have serious consequences for that individual.

How to prevent vulnerabilities

- Use HTTPS with a proper certificate and PFS (Perfect Forward Secrecy). Do not receive any information on non-HTTPS connections. Have a security flag on cookies.
- You need to limit the amount of sensitive data you have that could potentially be leaked. If you don't need this sensitive data, destroy it. Data you don't possess cannot be stolen.
- Never store credit card information if you want to avoid dealing with PCI compliance issues. Sign up for a payment processor like Stripe or Braintree.
- If you have sensitive data that you really need, store it encrypted and ensure that all passwords are protected using hash functions. For hashing, bcrypt is recommended. If you don't use bcrypt encryption, learn about salt encryption to prevent rainbow table attacks.
- Do not store encryption keys alongside protected data. This is like leaving your car keys in the ignition. Protect your backups with encryption and ensure your keys remain private.